

05

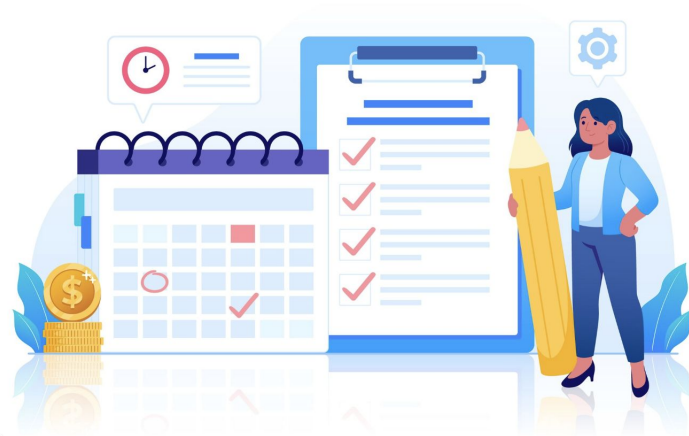
Lecture 05

**Thread Synchronization and Context
Switching**

by - Brijesh Shukla
C07: Operating Systems

Today's Agenda

- The "Lost Update" Problem
- Race Conditions & Synchronization
- Anatomy of a Context Switch
- Why thread switching is 5-10x faster.
- Scheduling & Overhead

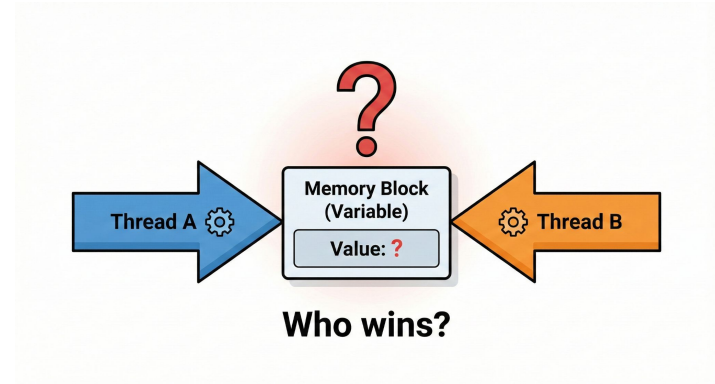


The Illusion of Concurrency

The Conflict: What happens when two threads access the same piece of data at the same time?

Good Synchronization: Google Docs. Two people type, cursor updates live, changes merge (mostly).

Race Condition: Two people open the same Excel file on a shared drive, edit it locally, and hit "Save." The last person to save wipes out the other's work.



Race Conditions & Synchronization

If two threads run this loop, what is the final value?

```
int counter = 0; // Global Shared

void *increment(void *arg) {
    for (int i=0; i < 1000000; i++)
    {
        counter++; // The Danger
Zone
    }
}
```

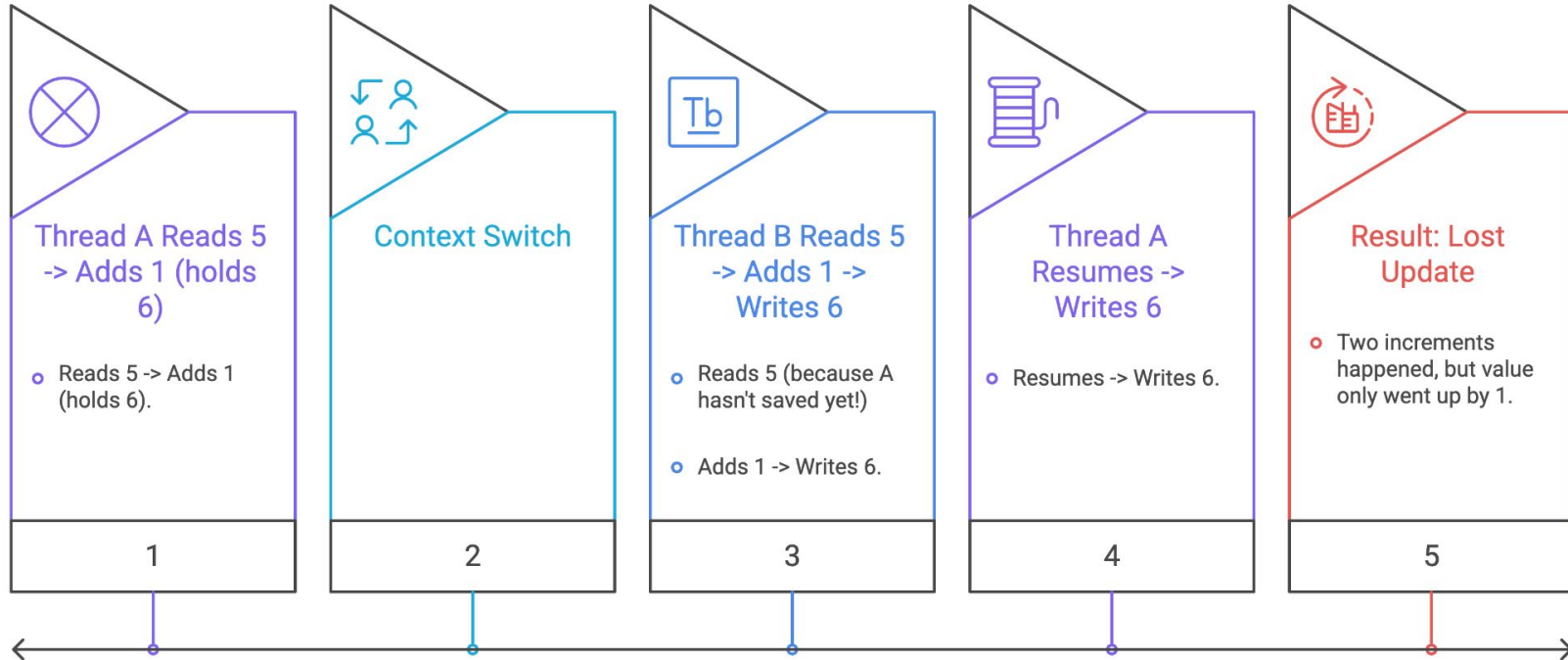
Expectation: **2,000,000.**

Reality: Likely **~1,450,293**
(Random).

<https://my.newtonschool.co/playground/code/gou73mwogigg>



Visualizing the Race



Characteristics of Race Conditions

- **Non-deterministic:** Different runs produce different results
- **Timing-dependent:** Depends on thread interleaving
- **Hard to debug:** May not occur during testing
- **Intermittent:** Bug appears randomly
- **Hardware-dependent:** May only show on multicore systems



Critical Sections & Mutual Exclusion

A Critical Section is the **specific lines of code** that access shared resources.

The Solution: **Mutual Exclusion (Mutex)**.

Safety: Only one thread inside at a time.

Analogy: The Airplane Bathroom

Only one person fits.

If the door shows "Occupied" (**Locked**), you wait in the aisle (**Block**).

When they leave, the sign turns "Vacant" (**Unlock**), and the next person enters.



The Mutex API (Pthreads)

Step 1: Declare the mutex

```
pthread_mutex_t lock;
```

Step 2: Initialize the mutex

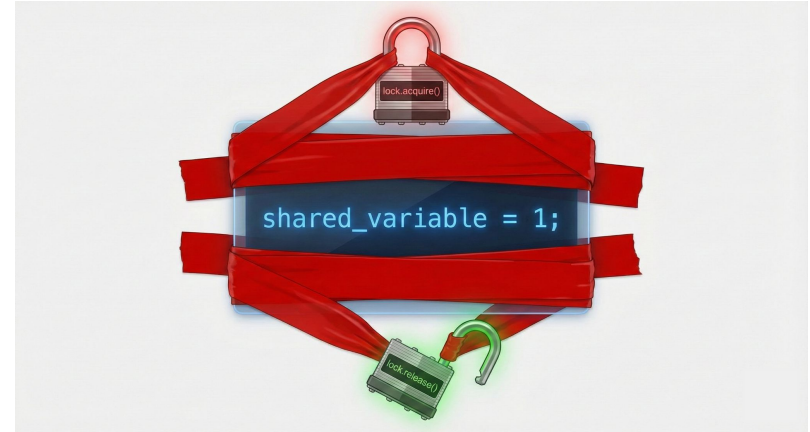
```
pthread_mutex_init(&lock, NULL);
```

Step 3: Protect the critical section

```
pthread_mutex_lock(&lock);           // LOCK  
// CRITICAL SECTION START  
counter++;  
// CRITICAL SECTION END  
pthread_mutex_unlock(&lock);         // UNLOCK
```

Step 4: Destroy the mutex when done

```
pthread_mutex_destroy(&lock);
```



Performance Note: Locking is safe, but slower.

Process Context Switching

What is a Context Switch?

The mechanism that allows the CPU to switch from running Process A to Process B.

The Trigger:

- Timer Interrupt (Preemptive).
- I/O Block (Voluntary).

The Cost: It is pure overhead. The system is doing administrative work, not "useful" work.

Anatomy of a Process Switch (The Heavy Lift)

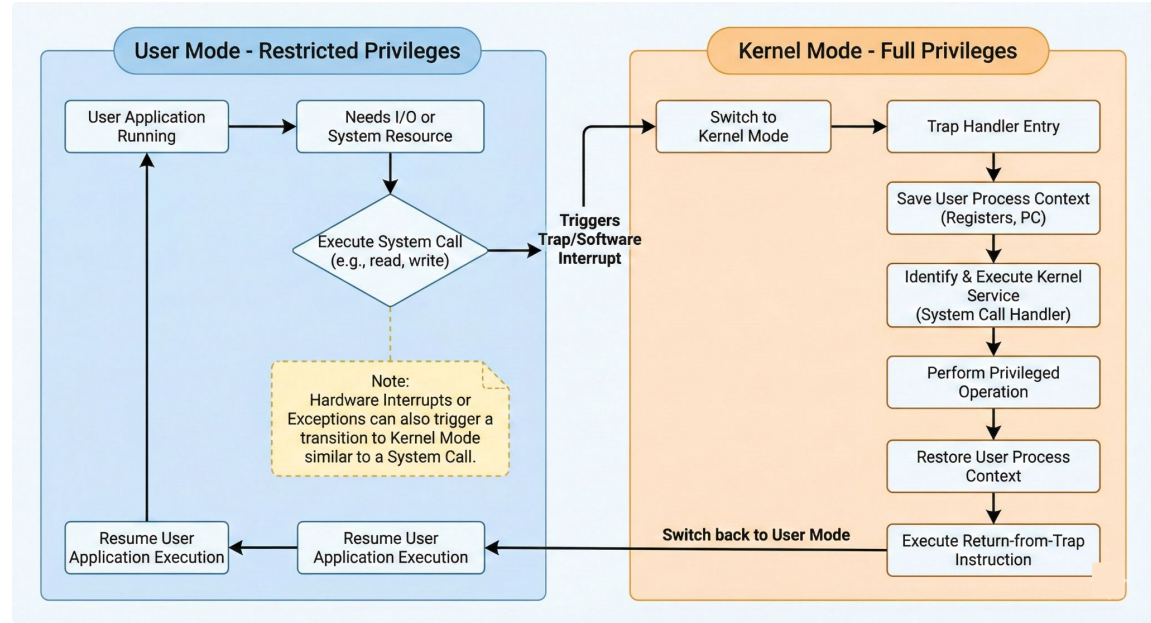
Trap/Interrupt: Switch to Kernel Mode.

Save State: Registers, PC, Stack Pointer saved to PCB (Process Control Block).

Scheduler: "Who runs next?"

Memory Switch: Switch Page Tables (CR3 register). <-- Expensive!

Restore State: Load new registers.



The Hidden Costs – TLB & Cache

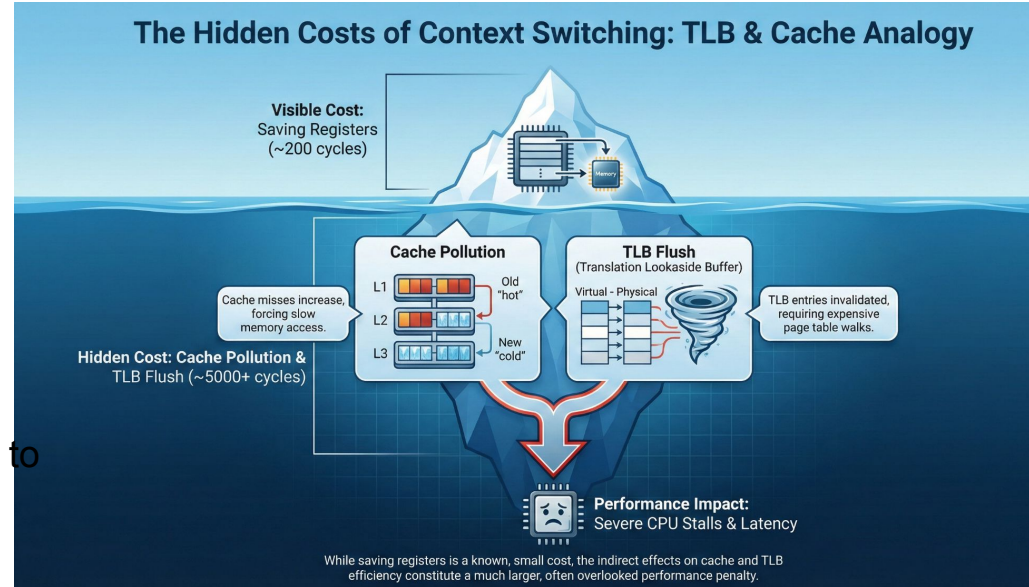
Visible Cost: Saving registers
(~200 cycles).

Invisible Cost: Cache Pollution &
TLB Flush (~5000+ cycles).

TLB (Translation Lookaside Buffer):

- Caches Virtual -> Physical memory addresses.
- Process A's addresses are meaningless to Process B.
- Result: Must FLUSH (empty) the TLB.

Consequence: The new process starts with a "cold" brain; every memory access is slow until the TLB refills.



Threads – The Lightweight Alternative

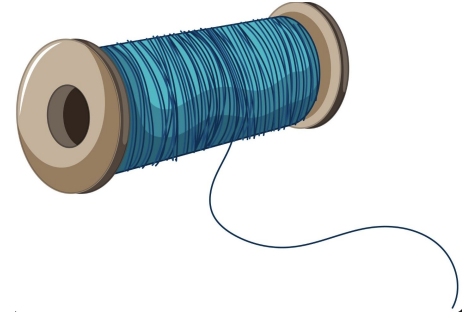
The Difference: Threads belong to the same process.

What they share (Don't need to switch):

- Page Tables (Address Space).
- TLB entries (Virtual addresses are still valid!).
- L3/L2 Cache data (mostly).

What they switch (Must save):

- Registers.
- Stack Pointer (Each thread has its own stack).
- Program Counter.



Comparison Table

Feature	Process Switch	Thread Switch (Same Proc)
Registers	Save/Restore	Save/Restore
Scheduler	Run Algorithm	Run Algorithm
Page Table	SWITCH	KEEP
TLB	FLUSH	KEEP
Cache	Cold (Polluted)	Warm (Shared Data)
Est. Time	~3-5 microseconds	~0.5 microseconds

Doing the Math

Scenario: CPU runs at 3GHz.

Process Switch: 3000 cycles.

Thread Switch: 1000 cycles.



Calculation:

If we switch threads 1000 times/sec:

$1000 * 1000 \text{ cycles} = 1 \text{ million cycles}$

Total cycles available: 3 Billion.

Overhead: **0.03% (Negligible).**

The Trap: What if we switch 100,000 times/sec? The overhead becomes significant.

Lesson: Context switching is fast, but not free. Avoid "Thrashing."

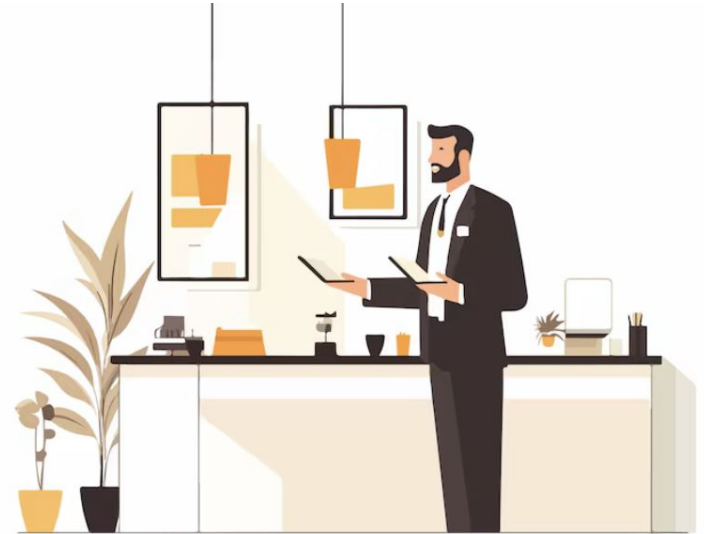
The Scheduler's Role

The Scheduler (The Brain/The Manager)

Role: This is pure software logic. It doesn't touch the hardware registers.

Action: It looks at the "Ready Queue" (the list of all threads waiting to run) and **runs an algorithm** (like Round Robin, Priority, or Shortest Job First) to pick **the Process Control Block (PCB)** of the next process.

Analogy: Think of a Restaurant Manager. The manager looks at the list of incoming orders and decides, "Okay, table 4 gets their food next because **they've been waiting the longest.**" The manager decides, but doesn't cook or carry the food.



The Dispatcher (The Muscle/The Worker)

Role: This is low-level assembly code. It executes the actual hardware manipulation.

Action: Once the Scheduler points to "Process B," the Dispatcher steps in to:

1. Save the current CPU registers to Process A's PCB.
2. Load the registers from Process B's PCB into the CPU.
3. Jump to the new program counter.

Analogy: This is the Waiter. Once the manager decides Table 4 is next, the waiter physically clears the old plates, wipes the table, and sets down the new food. This physical work takes time and energy.



The Trade-off: When to Switch?

This is the core design challenge of an Operating System. The Scheduler must decide the Time Quantum (**how long a process runs before it is removed**).

Scenario A: Switching Too Frequently (Small Time Quantum)

Example: The Scheduler switches every 1 ms.

The Problem (Overhead): If a context switch (Dispatcher work) takes 0.1 ms, and you switch every 1 ms, you are spending **10% of your total CPU power just switching tasks**.

Result: The computer feels "smooth" (high responsiveness), but heavy calculations take forever to finish because the CPU is wasting time "clearing the table" constantly.

Metric: **Low Throughput** (little actual work gets done).



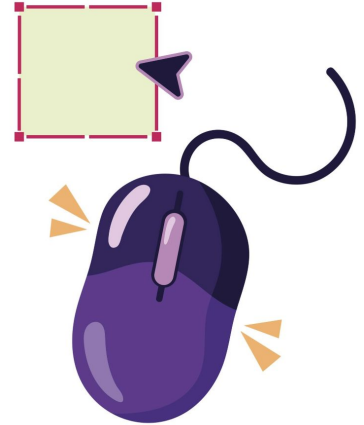
Switching Too Infrequently (Large Time Quantum)

Example: The Scheduler switches every 100 ms or 500 ms.

The Problem (Responsiveness): Imagine you move your mouse, but the CPU is busy crunching numbers for a video rendering app. If the scheduler waits 500ms before switching back to the "Mouse Driver" thread, your cursor will stutter or freeze.

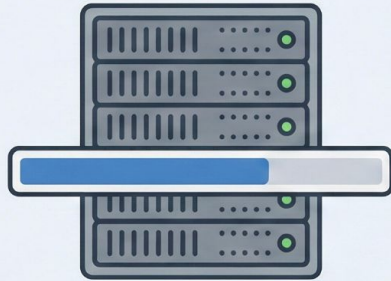
Result: The CPU is very efficient (it spends 99.9% of time working), but the user experience is terrible and laggy.

Metric: High Latency (long wait times).



Balance Scale (Throughput vs. Latency)

THROUGHPUT: Batch Processing



Scientific calculations,
video rendering

One big task for a long time,
minimize switching costs.

THE SWEET SPOT: Modern OSs



Dynamic Quantum
(10-20ms) OSs

Fast enough for human
eyes (<16ms usually),
low overhead (<1%).

LATENCY: Interactive Systems



UI, Gaming,
UI, Gaming, Web Servers

Switch constantly, no user
input ignored > a few ms.

Context Switch Trigger Code

```
void timer_interrupt_handler() {  
    // Save interrupted process state  
    save_context(&current->context);  
  
    // Update time accounting  
    current->quantum_remaining--;  
    current->cpu_time_used++;  
  
    // Check if quantum expired  
    if (current->quantum_remaining == 0) {  
        // Preempt: move to ready queue  
        enqueue_ready(current);  
  
        // Invoke scheduler  
        process_t *next =  
        scheduler_select_next();  
  
        // Dispatch  
        dispatch(next);  
    }  
  
    // Acknowledge interrupt and return  
    ack_interrupt();  
}
```

Summary

- Think of the Scheduler as the boss looking at a clipboard deciding who goes next
- The Dispatcher as the worker physically swapping the heavy machinery.
- If the boss changes their mind every 2 seconds, the worker spends all day swapping machines and no work gets done (High Overhead).
- If the boss never changes their mind, urgent tasks get ignored (High Latency)

Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.